Balancing Accuracy and Adaptability: Hybrid Analytical–Neural Control in Omnidirectional Robots

1st Sebastian Mendez-Villegas
Departamento de Estudios en Ingeniería para la Innovación
Universidad Iberoamericana
Mexico City, Mexico
a2229332@correo.uia.mx

2nd Julio Antonio Caballero-Mora
Departamento de Estudios en Ingeniería para la Innovación
Universidad Iberoamericana
Mexico City, Mexico
p42522@correo.uia.mx

3rd Alexandro López-González
Departamento de Estudios en Ingeniería para la Innovación
Universidad Iberoamericana
Mexico City, Mexico
alexandro.lopez@ibero.mx

4th Janet López-Romero
Departamento de Estudios en Ingeniería para la Innovación
Universidad Iberoamericana
Mexico City, Mexico
p41922@correo.uia.mx

5th Juan C. Tejada
Departamento de Estudios en Ingeniería para la Innovación
Universidad Iberoamericana
Mexico City, Mexico
juan.tejada@ibero.mx

***Abstract*—Three control strategies for navigation and trajectory tracking are presented and compared using the omnidirectional mobile robot *RoboMaster S1*, equipped with a *Vicon* motion capture system and controlled via *Python*. (1) Classic potential field control (P): the desired trajectory is modeled as an attractive field that generates the wheel velocities. (2) Neural network (NN) + proportional–integral (PI) control: a *feedforward* neural network is trained to predict wheel velocities from the desired trajectory, while an internal PI loop compensates for tracking errors. (3) Hybrid potential + NN control: the same potential field provides the reference, and a second neural network, trained on the residual error of the P method, supplies an online corrective signal (Δ). The main contributions of this work are: (i) the empirical validation of a neural model for velocity prediction in omnidirectional platforms; (ii) the proposal of a neural compensator integrated with analytical control laws; and (iii) a comparative experimental analysis of potential-based, neural, and hybrid control approaches.**

***Index Terms*—component, formatting, style, styling, insert.**

I. Introduction

Omnidirectional robots offer superior maneuverability in constrained environments, as they can move in any direction without changing their orientation [1], [2]. This capability makes them particularly advantageous in service, logistics, and autonomous exploration applications. Nevertheless, the kinematics of these platforms are highly coupled and exhibit nonlinearities caused by friction, mechanical backlash, actuator saturations, and dead zones [3], [4]. Such effects complicate the design of control laws that ensure tracking accuracy, smooth motion, and robustness against disturbances in dynamic scenarios.

Among classical techniques, potential fields have been widely applied to reactive navigation and trajectory tracking due to their computational simplicity and the intuitive physical interpretation of attractive and repulsive forces [5], [6]. However, their performance degrades when the gain parameters are not properly tuned to operating conditions or when the model fails to account for external disturbances and inherent robot limitations. To improve robustness, proportional–integral–derivative (PID) controllers have been employed to compensate for tracking errors [7], but these schemes require manual fine-tuning and may induce oscillations in the presence of delays or actuator saturations [8].

In contrast, neural network (NN)–based methods have demonstrated strong ability to approximate complex nonlinear dynamics and to generate control policies directly from the desired trajectory [9], [10]. These policies are trained from experimental data and can adapt to variations in system dynamics. However, they often lack analytical stability guarantees, and their performance may deteriorate when operating outside the training domain [11], [12].

More recently, the fusion of analytical and learning techniques has emerged as a promising strategy to combine the interpretability and stability of classical controllers with the adaptability of neural networks (NNs) [13], [14]. In particular, hybrid schemes allow an analytical controller (e.g., potential

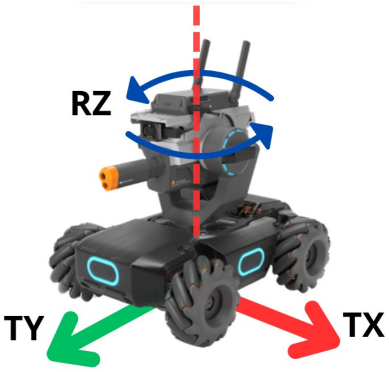


Fig. 1: RoboMaster S1 - DJI

fields or PID) to guarantee nominal stability, while a NN functions as an uncertainty or residual error compensator. This synergy is especially relevant in omnidirectional platforms, where backlash and slip effects can significantly degrade performance.

II. Methodology

A. Experimental platform

The experiments were carried out on an omnidirectional mobile robot, the RoboMaster S1, developed by Da-Jiang Innovations (DJI). The vehicle is equipped with four Mecanum wheels that allow motion in any direction through differential speed control. Specifically, the *S1* can perform translation along the X axis (Tx), translation along the Y axis (Ty), and rotation around its own Z axis (Rz) with respect to an XYz reference frame (Fig. 1). Thanks to its omnidirectional nature, it can also execute simultaneous combinations of these movements [2].

The absolute position and orientation of the platform were measured using a *Vicon* motion capture system composed of seven cameras (models *Bonita* and *Vero*) operating at 100 Hz. The rigid body pose $\mathbf{x} = [x, y, \theta]^\top$ was transmitted in real time through the platform's SDK via the User Datagram Protocol (UDP), enabling direct integration with the control loop [15].

The controller operated at an effective frequency of 10 Hz, which was sufficient to capture the dynamics of the *S1* and to ensure stable data exchange with the motion capture system [3].

B. Potential Field Control

1) Formulation of the Field and P Law: The attractive field is defined in (1):

$$\mathbf{F}(\mathbf{x}) = -K_p\,(\mathbf{x} - \mathbf{x}_d), \tag{1}$$

where $\mathbf{x}_d(t)$ denotes the desired trajectory and $K_p = \mathrm{diag}\{k_x,\, k_y,\, k_\theta\}$ is a proportional gain matrix. The resulting force $\mathbf{F}$ is mapped to the desired body velocities $\mathbf{v}_d = [v_x,\, v_y,\, \omega]^\top$ through a linear saturation function, ensuring bounded control inputs compatible with the platform actuators.

C. Gain Adjustment

The initial gains were set to $k_x = k_y = 4$ and $k_\theta = 3.9$, values obtained by trial and error in order to minimize overshoot. Subsequently, the gains were fine-tuned in increments of 0.1 until the position error along a circular trajectory of radius $r = 1.0$ m was reduced to less than 0.025 m.

D. Neural Network with PI Loop

1) Training Trajectory Design: To ensure coverage of the feasible velocity space, a concatenated sequence of motion primitives was programmed, consisting of:

1) translation along the **X**-axis (two straight segments),
2) diagonals at 45° (two straight segments),
3) intermediate diagonals between the previous ones (four straight segments).

The resulting pattern had a total duration of 96 s and explored body velocities within the ranges $|v_x|, |v_y| \leq 0.7$ m/s and $|\omega| \approx 0$ rad/s. The complete set of predefined motion segments used in Block I is illustrated in Fig. 2.

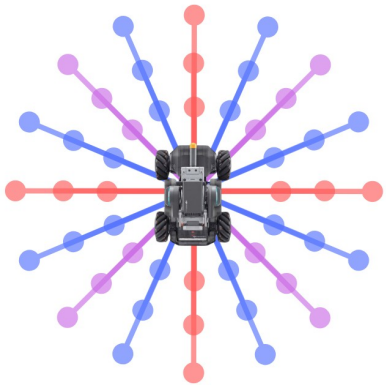


Fig. 2: Predefined motion patterns used in Block I.

2) Data Collection and Inversion: During execution, data pairs $(\mathbf{v}_{\text{body}}, \boldsymbol{\omega}_{\text{wheels}})$ were recorded, where $\mathbf{v}_{\text{body}}$ was obtained from finite differences of the Vicon pose, and $\boldsymbol{\omega}_{\text{wheels}}$ from the internal encoders of the *S1*. A total of approximately 12,000 samples were collected, of which 80% were used for training and 20% for validation.

3) Network Architecture and Training: The predictive model consisted of a two–hidden-layer multilayer perceptron (MLP) with 64 and 32 neurons, respectively, and *ReLU* activation functions. The network was implemented in *PyTorch 2.1* and trained using mean squared error (MSE) loss with the *Adam* optimizer ($\eta = 10^{-3}$, batch size = 256). Training converged after 150 epochs with early stopping (patience = 10). The final model occupied 19 kB and achieved an inference time of 0.12 ms on CPU.

4) PI Control Implementation: The neural policy generated the nominal wheel speeds $\hat{\boldsymbol{\omega}}$. The body velocity error was defined by (2):

$$\mathbf{e}_{\text{PI}} = \mathbf{v}_d - \mathbf{v}_{\text{body}}, \tag{2}$$

and compensated through a PI loop (3):

$$\mathbf{u}_{\text{PI}} = K_p^{\text{PI}}\,\mathbf{e}_{\text{PI}} + K_i^{\text{PI}} \int \mathbf{e}_{\text{PI}}\,dt, \tag{3}$$

where $K_p^{\text{PI}} = \text{diag}(2,\ 2,\ 1)$ and $K_i^{\text{PI}} = \text{diag}(1,\ 1,\ 0.4)$. The final wheel command was then computed in (4).

$$\boldsymbol{\omega} = \hat{\boldsymbol{\omega}} + \mathbf{u}_{\text{PI}}. \tag{4}$$

E. Potential Hybrid Control + Δ-NN

1) Error Dataset Generation: To construct the residual dataset, the pure potential field controller was executed on a circular trajectory of radius $r = 1.0$ m. At each instant, the residual term was computed in (5).

$$\Delta(t) = \boldsymbol{\omega}_{\text{real}}(t) - \boldsymbol{\omega}_{\text{pot}}(t), \tag{5}$$

together with the potential-based command $\boldsymbol{\omega}_{\text{pot}}(t)$ and the tracking error $\mathbf{e}(t)$. A total of 9,600 samples were collected for training the compensator network.

2) Corrective Network Training: A second multilayer perceptron (MLP) with two hidden layers (32 and 16 neurons, *ReLU* activations) was trained to receive as input the pair $[\mathbf{e},\ \boldsymbol{\omega}_{\text{pot}}]$ and to predict the residual signal Δ. The training was carried out using mean squared error (MSE) loss and the *Adam* optimizer with a learning rate $\eta = 2 \times 10^{-3}$, over 120 epochs, and incorporating a dropout rate of 0.1 to improve generalization.

3) Control Signal Fusion: In the hybrid control loop, the final wheel command was computed by (6).

$$\boldsymbol{\omega} = \boldsymbol{\omega}_{\text{pot}} + \Delta_{\text{NN}}(\mathbf{e}, \boldsymbol{\omega}_{\text{pot}}), \tag{6}$$

where Δ_{NN} denotes the corrective signal generated by the trained network. The resulting command was sent directly to the robot's internal speed controllers. To mitigate the risk of instability, the corrective term was attenuated by a factor of 0.3, i.e., $\Delta_{\text{NN}} \leftarrow 0.3 \cdot \Delta_{\text{NN}}$.

F. Signal Filtering and Derived Variables

The captured position signals were smoothed using a second-order Butterworth filter, with cutoff frequencies of 1.1 Hz for x, y and 0.4 Hz for θ. Body velocities (V_x, V_y, ω) and accelerations (A_x, A_y, α) were then computed using central differences. The resulting cleaned dataset was stored in a CSV file for later use.

G. PI Control Loop + Inverse Network

The control loop was executed every 20 ms (50 Hz) according to the following sequence:

1) The position (x, y) was read and the error $\mathbf{e} = (e_x, e_y)$ was calculated.
2) A PI law with $k_p = 2.0$ and $k_i = 0.4$ generated the commands V_x^{cmd} and V_y^{cmd}.
3) The angular velocity command was fixed at $\omega^{cmd} = 0$ rad/s, and the acceleration terms were set to zero.
4) The command vector $\left(V_x^{cmd}, V_y^{cmd}, \omega^{cmd}, 0, 0, 0\right)$ was passed to the inverse network, and the resulting PWM signals were saturated to ± 100 rpm.

The total latency of the loop, including PI computation, neural inference, and transmission, was approximately 4 ms.

III. Results

We evaluated five scenarios: (E1) **NN+PI** (one run); (E2–E3) **Potential fields** with two *initial conditions*: *aligned* ($\theta_0 = 0$) and *rotated* ($\theta_0 \approx -\pi/4$); and (E4–E5) **Potential + ΔNN (hybrid)** with the same two conditions. In all cases, the initial position was set at the origin, and the reference trajectory was a circle of radius $1.0\,\text{m}$.

A. E1: NN+PI

The neural policy assisted by a **proportional–integral (PI)** loop under the *aligned start* condition successfully completed the circular trajectory, but with a **higher radial error** ($e_r = 0.132\,\text{m}$) compared to the analytical methods, as shown in Fig. 3. Relative to the hybrid controller under the same condition, the hybrid reduced the radial error by approximately **35%** (from 0.132 to $0.086\,\text{m}$). The average orientation bias was $2.995°$, comparable to that observed in the aligned hybrid case. The evolution of the control variables and the corresponding tracking errors for this experiment are shown in Fig. 4.

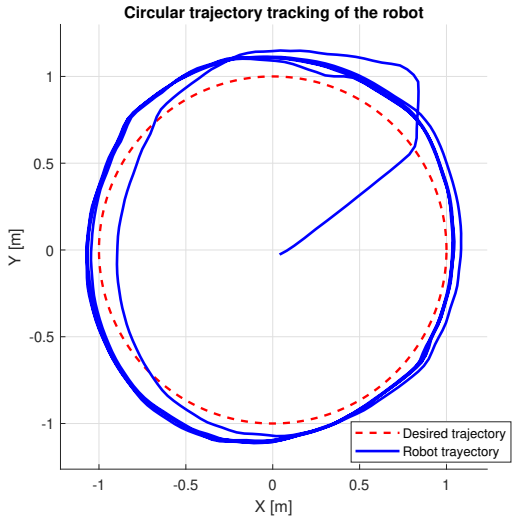


Fig. 3: Robot trajectory tracking using NN+PI (E1).

Video: **Neural Network Aligned**

B. E2–E3: Potential Fields

With the *aligned start* condition (E2), the potential field controller achieved $e_r = 0.0868\,\text{m}$ and an average orientation bias of $6.459°$. Under the *rotated start* condition (E3), the radial error **improved** to $e_r = 0.0805\,\text{m}$ (approximately **7.3%** lower than in the aligned case), although a noticeable orientation bias of $-3.98°$ remained. Trajecory tracking is shown in Fig. 5 and error evolution for both initial conditions are shown in Fig. 6.

Video: **Potential Aligned**

Video: **Potential Rotated**

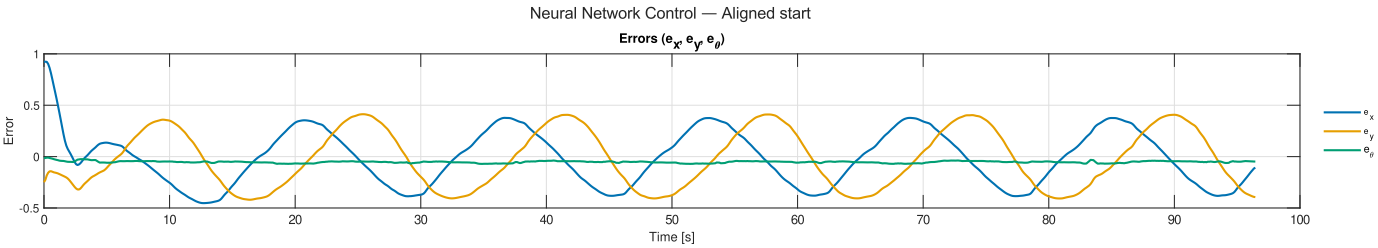


Fig. 4: E1 (NN+PI, aligned start). Evolution of tracking errors. Colors: X (blue), Y (orange), θ (green).

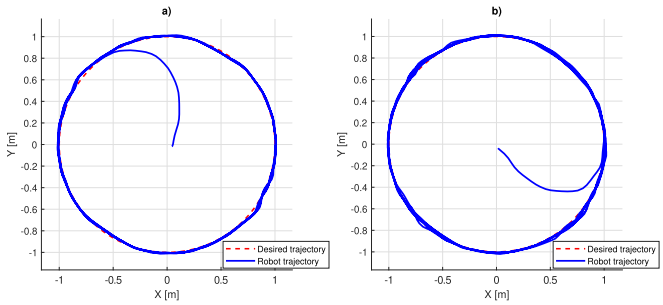


Fig. 5: Robot trajectory tracking using Potential fields: a) E2 *aligned*: ($\theta_0 = 0$) and b) E3 *rotated* ($\theta_0 \approx -\pi/4$).

TABLE I: Experimental summary (radial RMSE and average orientation).

Exp.	Initial condition	e_r [m]	$\langle\theta\rangle$ [rad / °]
E1: NN+PI	aligned	0.132047	0.052265 / 2.995°
E2: Potential	aligned	0.086832	0.112738 / 6.459°
E3: Potential	rotated	0.080539	−0.069458 / − 3.980°
E4: Hybrid	aligned	0.086017	0.037202 / 2.132°
E5: Hybrid	rotated	0.084606	0.015238 / 0.873°

C. E4–E5: Hybrid Potential + ΔNN

With the *aligned start* condition (E4), the hybrid controller achieved $e_r = 0.0860\,\mathrm{m}$ and reduced the **orientation bias** to $2.132°$, representing an approximate **67% decrease** compared to the aligned potential case. Under the *rotated start* condition (E5), the radial error was $e_r = 0.0846\,\mathrm{m}$ (slightly higher than the rotated potential case by ∼5%), but the **bias** decreased to $0.873°$, an improvement of about **78%** relative to the rotated potential case. These results indicate that the corrective term Δ_{NN} effectively mitigates orientation drift without significantly penalizing radial accuracy. The trajectory tracking and error evolution for both conditions are depicted in Fig. 7 and Fig. 8 respectively.

Video: **Hybrid Aligned**

Video: **Hybrid Rotated**

D. Comparative Discussion

Two performance metrics were considered: (i) the **radial RMSE** (7), which evaluates tracking accuracy with respect to the reference radius calculated ; and (ii) the **average orientation** $\langle\theta\rangle$ during the test (sign according to the established convention). These metrics jointly capture trajectory tracking performance and orientation bias. The average orientation was computed using the *circular mean* to properly account for the periodicity of angular data. The orientation metric is defined in (8).

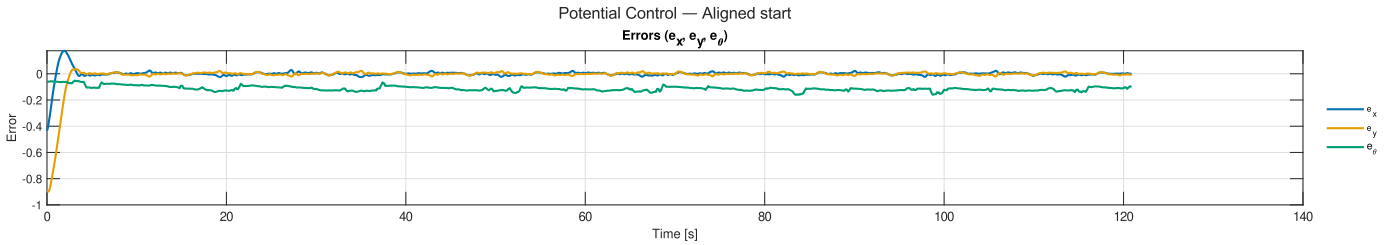
$$e_r = \sqrt{\tfrac{1}{T}\sum_t \left(\,\|[x(t)\;\; y(t)]\| - r\,\right)^2}, \quad r = 1.0\,\mathrm{m}, \qquad (7)$$

$$\langle\theta\rangle = \operatorname{atan2}\left(\frac{1}{N}\sum_{i=1}^{N}\sin(\theta_i),\ \frac{1}{N}\sum_{i=1}^{N}\cos(\theta_i)\right), \qquad (8)$$

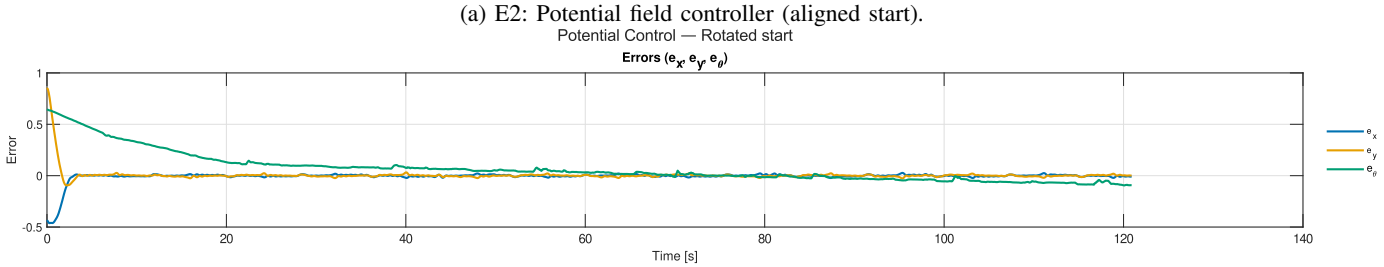
where θ_i denotes each instantaneous orientation measurement and N is the total number of samples. This formulation avoids discontinuities that occur with direct arithmetic averaging of angles near $\pm\pi$.

The NN+PI controller (E1) achieved stable tracking but produced the largest radial error ($e_r = 0.132$ m) among the tested strategies, reflecting the limited corrective capacity of the PI loop when combined with the learned policy. In contrast, the pure potential field controllers (E2–E3) achieved lower radial errors ($e_r \approx 0.08$ m) but exhibited a significant orientation bias: positive when starting aligned (E2, $\langle\theta\rangle = 6.46°$) and negative when starting rotated (E3, $\langle\theta\rangle = -3.98°$). The hybrid controllers (E4–E5) preserved the low radial error of the potential method while considerably reducing orientation bias, especially in the rotated case (E5, $\langle\theta\rangle = 0.87°$). These results indicate that the hybrid scheme effectively combines the accuracy of potential fields with the compensatory capabilities of neural networks, producing smoother and more balanced trajectories across different initial conditions.

In terms of radial precision, the **best case** was obtained with the Potential controller under the *rotated start* condition ($e_r = 0.0805\,\mathrm{m}$). The **hybrid controller** achieved nearly the same performance in both conditions ($0.0846\,\mathrm{m}$ to $0.0860\,\mathrm{m}$) and consistently outperformed the potential controller in terms of orientation bias (reductions of $67\,\%$ and $78\,\%$ in the aligned and rotated cases, respectively). The **NN+PI** approach showed the largest e_r ($0.132\,\mathrm{m}$), reinforcing the advantage of *anchoring* a learned policy to an analytical model (hybrid scheme) to improve stability and overall performance.



(a) E2: Potential field controller (aligned start).



(b) E3: Potential field controller (rotated start).

Fig. 6: Error evolution with the potential field controller. Colors: X (blue), Y (orange), θ (green).

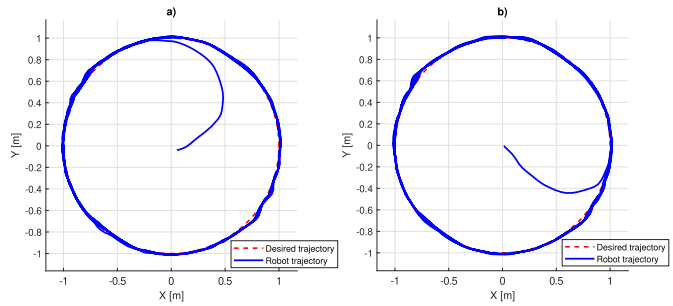


Fig. 7: Robot trajectory tracking using Hybrid Potential + ΔNN: a) E4 *aligned*: ($\theta_0 = 0$) and b) E5 *rotated* ($\theta_0 \approx -\pi/4$).

Table I summarizes the experimental performance across the five scenarios. The results indicate that the pure potential field controller (E3) achieved the best radial accuracy with an e_r of 0.080539 m. However, the Hybrid Potential + ΔNN controller (E5) delivered the most balanced performance, achieving the lowest orientation bias with an average $\langle\theta\rangle$ of $0.873°$, which represents an improvement of about 78% relative to the rotated potential case.

IV. Conclusions

This work presented a comparative evaluation of three control strategies for an omnidirectional mobile robot: (i) a neural network policy assisted by a PI loop (NN+PI), (ii) a classical potential field controller, and (iii) a hybrid scheme that integrates potential fields with a corrective neural network (ΔNN). The RoboMaster S1 platform, combined with a Vicon motion capture system, enabled precise tracking and validation of the proposed methods.

The experimental results demonstrate that the NN+PI controller can successfully complete the trajectory but exhibits the largest radial error, highlighting the limited corrective capacity of a purely data-driven policy when not anchored to analytical models. In contrast, the pure potential field controller achieves low radial errors but suffers from orientation drift, which is strongly dependent on the initial condition. The hybrid controller preserved the accuracy of the potential approach while significantly reducing orientation bias in both aligned and rotated starts. This outcome shows that residual learning effectively compensates for unmodeled dynamics and backlash effects without compromising radial precision.

Overall, the results confirm that combining analytical control with learning-based compensation leverages the strengths of both approaches: stability and interpretability from potential fields, and adaptability from neural networks. The hybrid method yielded smoother and more balanced trajectories, demonstrating its suitability for navigation tasks in omnidirectional platforms operating under real-world uncertainties.

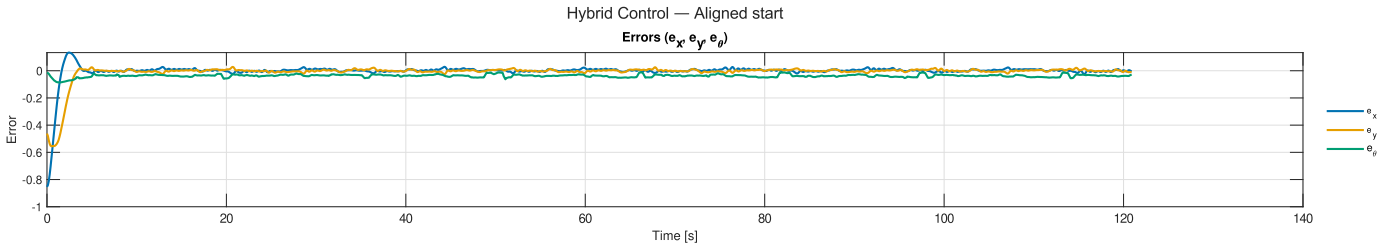
Future work will explore extending the hybrid approach to trajectories with higher angular velocities, incorporating online adaptation to unseen conditions, and validating the method in multi-robot scenarios where interaction dynamics further challenge controller robustness.

Acknowledgment

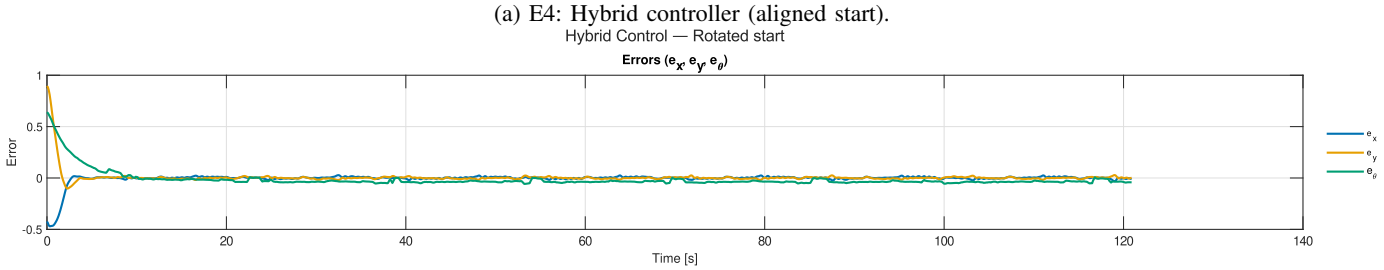
The authors gratefully acknowledge the financial support from Universidad Iberoamericana Ciudad de Mexico and SECIHTI, for grant number 1228748, 809071 and 1054065.

References

[1] G. Campion, G. Bastin, and B. D'Andrea-Novel, "Structural properties and classification of kinematic and dynamic models of wheeled mobile robots," *IEEE Transactions on Robotics and Automation*, vol. 12, no. 1, pp. 47–62, 1996. DOI: 10.1109/70.481750.



(a) E4: Hybrid controller (aligned start).



(b) E5: Hybrid controller (rotated start).

Fig. 8: Error evolution with the hybrid controller. Colors: X (blue), Y (orange), θ (green).

[2] R. Siegwart, I. R. Nourbakhsh, and D. Scaramuzza, *Introduction to Autonomous Mobile Robots*, 2nd ed. MIT Press, 2011, ISBN: 978-0-262-01535-6.

[3] B. Siciliano, L. Sciavicco, L. Villani, and G. Oriolo, *Robotics: Modelling, Planning and Control* (Advanced Textbooks in Control and Signal Processing). London: Springer, 2010, ISBN: 9781846286414.

[4] T. Oren and F. Branz, "Modeling of wheel-ground interaction for mobile robots with omnidirectional wheels," *Journal of Field Robotics*, vol. 27, no. 3, pp. 155–170, 2010. DOI: 10.1002/rob.20323.

[5] O. Khatib, "Real-time obstacle avoidance for manipulators and mobile robots," *The International Journal of Robotics Research*, vol. 5, no. 1, pp. 90–98, 1986. DOI: 10.1177/027836498600500106.

[6] S. S. Ge and Y. J. Cui, "Dynamic motion planning for mobile robots using potential field method," *Autonomous Robots*, vol. 13, no. 3, pp. 207–222, 2002. DOI: 10.1023/A:1019621407015.

[7] K. H. Ang, G. Chong, and Y. Li, "Pid control system analysis, design, and technology," *IEEE Transactions on Control Systems Technology*, vol. 13, no. 4, pp. 559–576, 2005. DOI: 10.1109/TCST.2005.847331.

[8] K. J. Å ström and T. Hägglund, *PID Controllers: Theory, Design, and Tuning*, 2nd ed. Instrument Society of America, 1995, ISBN: 978-1-55617-516-9.

[9] Y. LeCun, Y. Bengio, and G. Hinton, "Deep learning," *Nature*, vol. 521, no. 7553, pp. 436–444, 2015. DOI: 10.1038/nature14539.

[10] J. Schmidhuber, "Deep learning in neural networks: An overview," *Neural Networks*, vol. 61, pp. 85–117, 2015. DOI: 10.1016/j.neunet.2014.09.003.

[11] J.-J. E. Slotine and W. Li, *Applied Nonlinear Control*. Prentice Hall, 1991, ISBN: 978-0130408907.

[12] M. Raissi, P. Perdikaris, and G. E. Karniadakis, "Physics-informed neural networks: A deep learning framework for solving forward and inverse problems involving nonlinear partial differential equations," *Journal of Computational Physics*, vol. 378, pp. 686–707, 2019. DOI: 10.1016/j.jcp.2018.10.045.

[13] N. Nguyen and H. La, "Review of deep reinforcement learning for robot navigation," in *2018 IEEE International Conference on Robotic Computing (IRC)*, 2018, pp. 383–388. DOI: 10.1109/IRC.2018.00072.

[14] F. L. Lewis, S. Jagannathan, and A. Yesildirak, *Neural Network Control of Robot Manipulators and Nonlinear Systems*. CRC Press, 2004, ISBN: 978-0824753340.

[15] Vicon, *Vicon motion systems documentation*, 2010. [Online]. Available: https://docs.vicon.com/.